iMScIT Programme- Sem-6

SOFTWARE TESTING
221604603

Introduction To Bug, Defect & Failure

(UNIT 4)

What is a Bug?

- Bug is an error detected in the development environment during testing stage.
- When we have any type of logical error, it causes our code to break, which results in a bug.
- It is now that the Automation/ Manual Test Engineers describe this situation as a bug.
- A bug once detected can be reproduced with the help of standard bug-reporting templates.
- Major bugs are treated as prioritized and urgent especially when there is a risk of user dissatisfaction.
- Typos are also bugs that seem tiny but are capable of creating disastrous results.

What is a Defect?

- A defect refers to a situation when the application is not working as per the requirement and the actual and expected result of the application or software are not in sync with each other.
- The defect is an issue in application coding that can affect the whole program.
- It represents the efficiency and inability of the application to meet the criteria and prevent the software from performing the desired work.
- The defect can arise when a developer makes major or minor mistakes during the development phase.

What is a Fault?

- Sometimes due to certain factors such as Lack of resources or not following proper steps Fault occurs in software which means that the logic was not incorporated to handle the errors in the application.
- This is an undesirable situation, but it mainly happens due to invalid documented steps or a lack of data definitions.
- It is an unintended behavior by an application program.
- It causes a warning in the program.
- If a fault is left untreated it may lead to failure in the working of the deployed code.
- A minor fault in some cases may lead to high-end error.
- There are several ways to prevent faults like adopting programming techniques, development methodologies, peer review, and code analysis.

What is a Failure?

- Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information in critical modules thereby making the system unresponsive.
- Generally, such situations happen very rarely because before releasing a product all possible scenarios and test cases for the code are simulated.
- Failure is detected by end-users once they face a particular issue in the software.
- Failure can happen due to human errors or can also be caused intentionally in the system by an individual.
- It is a term that comes after the production stage of the software.
- It can be identified in the application when the defective part is executed.

Bug V/s Defect V/s Fault V/s Failure

Basis	Bug	Defect	Fault	Failure
Definition	A bug refers to defects which means that the software product or the application is not working as per the adhered requirements set	A Defect is a deviation between the actual and expected output	A Fault is a state that causes the software to fail and therefore it does not achieve its necessary function.	Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information in critical modules thereby making the system unresponsive.
Raised by	Test Engineers	The defect is identified by The Testers And is resolved by developers in the development phase of SDLC.	Human mistakes lead to fault.	The failure is found by the test engineer during the development cycle of SDLC

Bug V/s Defect V/s Fault V/s Failure

Basis	Bug	Defect	Fault	Failure
Reasons behind	• Missing Logic • Erroneous Logic • Redundant codes	• Receiving & providing incorrect input • Coding/Logical Error leading to the breakdown of software	• Wrong design of the data definition processes. • An irregularity in Logic or gaps in the software leads to the non-functioning of the software.	• Environment variables • System Errors • Human Error

Defect Management In Testing - Introduction

- Defects are basically considered as destructive in all software development stages.
- There is no such software that is present without any defect.
- Defects are present in whole life of software because software is developed by humans and “to err is human” i.e. it is natural for human beings to make mistakes.
- Number of defects can be reduced by resolving or fixing but it is impossible to make a software error or defect-free.
- It’s impossible to create software completely free of errors, but you can reduce the number of bugs by fixing them.

Defect Management In Testing - Introduction

- Defect Management Process (DMP), as name suggests, is a process of managing defects by simply identifying and resolving or fixing defects.
- If defect management process is done in more efficient manner with full focus, then less buggy software will be available more in the market.
- The defect management process (DMP) is integral to software testing. It's all about finding and fixing bugs, not just for the testing team but for everyone involved in the software development.
- The defect management process focuses mainly on preventing defects, finding them and fixing them early, and minimizing their impact.
- Tools like JIRA, Bugzilla, or Azure DevOps are used to log and track defects.

Importance Of Defect Management In Testing

- Identifying and rectifying bugs improves the overall quality of the software. This leads to a better user experience and higher customer satisfaction.
- Unresolved defects can lead to system failures, security breaches, or other critical issues. Having an effective defect management process in place helps mitigate these risks by identifying and addressing problems early in the development process.
- A well-structured defect management process provides valuable feedback for continuous improvement. It allows teams to analyze the cause of the issue and helps prevent similar issues from occurring in the future.

Importance Of Defect Management In Testing

- It helps eliminate false positives caused by anomalies in the test environment, issues with test code or data, or test process flaws.
- Removing duplicate entries that can occur when a bug creates results that seem like separate, unrelated problems.
- It helps ensure that people will still report defects promptly, even if some items are taken off the list.

Defect Life Cycle

- Defect has following 7 phases in its life cycle:
- **Identification**: Detects and confirms the existence of defects.
- **Logging**: Ensures defects are systematically recorded for tracking and resolution.
- **Classification**: Helps in prioritizing and categorizing defects for efficient handling.
- **Assignment**: Ensures accountability by assigning defects to the appropriate person or team.
- **Fixing**: Resolves the defect and restores intended functionality.
- **Retesting**: Validates the resolution and ensures no regressions occur.
- **Closure**: Confirms that the defect is resolved and marks the end of the defect life cycle.

Defect Life Cycle

- **1) Identification**:
- We can discover defects during various stages of software development, such as design, coding, testing, or even after the software has been released.
- For example, during the testing phase, a tester may encounter a functionality issue or unexpected software behavior, which indicates a defect.
- The tester replicates the issue to confirm it is a valid defect and documents it with clear evidence, such as steps to reproduce, screenshots, or logs.

Defect Life Cycle

- **2) Logging:**
- The defect is recorded in a defect tracking tool (e.g., JIRA, Bugzilla, or Azure DevOps).
- Information logged includes:
  - Defect ID.
  - Title and Description.
  - Steps to reproduce.
  - Expected vs. actual results.
  - Severity (critical, major, minor, trivial).Priority (high, medium, low).
  - Environment details (browser, OS, version).
  - Attachments (e.g., screenshots, videos, or logs).

Defect Life Cycle

- **3) Classification:**
- In this phase, the project manager reviews and assigns the defect's severity and priority.
- The defect is classified based on:
  - Type: Functional, performance, security, UI, etc.
  - Severity: Impact on the system or end-user.
  - Priority: Urgency of resolving the defect.
- For example, a critical defect with a high priority may need to be fixed immediately to avoid causing significant damage to the system or the end-user experience.

Defect Life Cycle

- **Assignment:**
- The defect is assigned to a developer or a responsible team for further investigation and resolution.
- At this stage, the defect status changes to "Assigned".
- The developer receives all the necessary information about the defect, such as its reproduction steps and the expected behavior to resolve the issue.

Defect Life Cycle

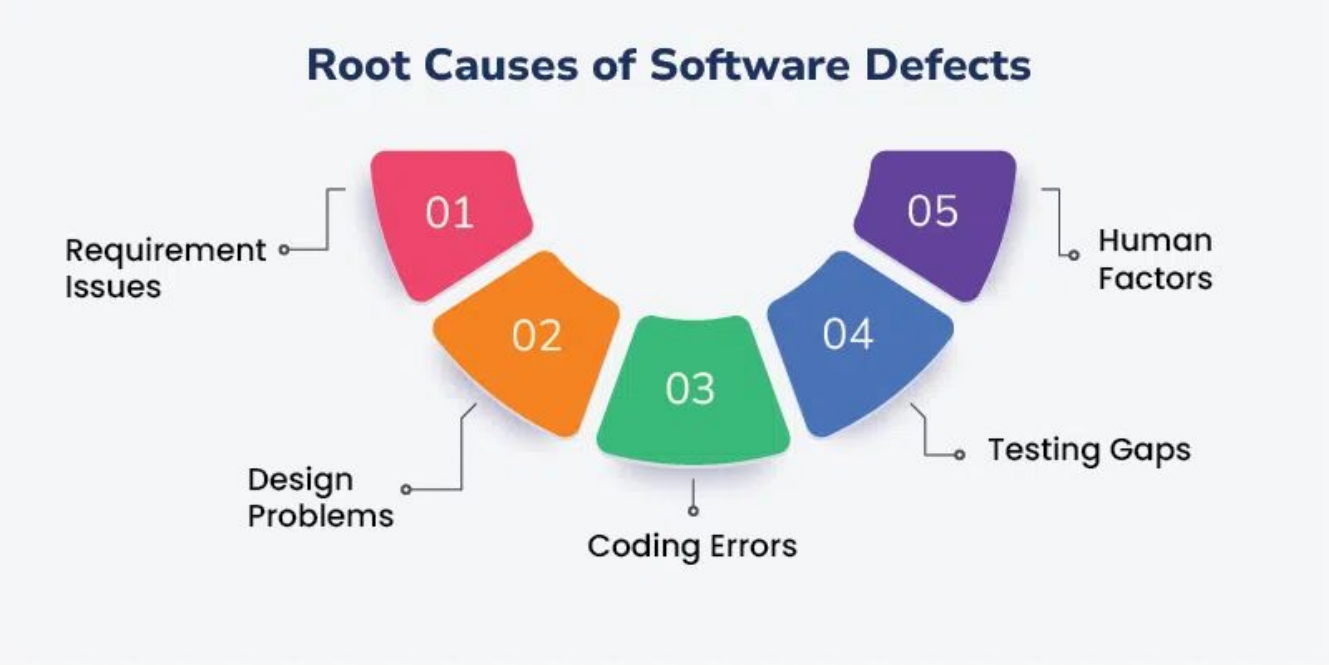
- **Fixing:**
- The developer works on analyzing and resolving the defect.
- They update the defect tracking system with details about the fix.
- Once the fix is complete, the status changes to "Fixed" and it is sent back to the testing team.
- **Retesting**
- The testing team verifies the fix by re-executing the test case(s) that caused the defect.
- If the defect is resolved, it moves to the next stage.
- If not, the status changes to "Reopened" and it is sent back to the developer.

Defect Life Cycle

- **Closure:**
- Once the defect has been verified, it is marked as “Closed” in the defect tracking tool.
- The defect is considered closed when it meets the criteria for closure, such as passing all the related test cases and receiving approval from the project manager.
- Closure includes updating documentation and ensuring the fix does not introduce new defects.

Causes For Defect

- Understanding the root causes of software defects is the first step towards preventing them. Common root causes include:



Causes For Defect – 1) Requirement Issues

- **Ambiguous or Incomplete Requirements:** Lack of specificity or absence of clear instructions will result in confusion and the wrong approach being taken.
- When the requirements are not well spelled out, the developers may understand them in a different way hence developing something that does not suit the users.
- This leads to redoing of work and time wastage which accounts to a lot of problems.
- **Changing Requirements:** If there are too many changes to requirements, they become confusing and mistakes happen. Where there are constant changes in requirements it becomes very difficult for individuals to work on projects, this creates confusion and very incomplete implementations.

Causes For Defect – 2) Design Problems

- **Poor System Architecture:** Lack of good design and/or design flaws ensures one experiences poor performance and problems with maintenance.
- They include scalability issues, insecurity, and increased difficulties in development in future with a weak foundation.
- **Insufficient Design Reviews:** They feared that if the design was flawed it could escalate to the development phase without proper design reviews.
- If the designs are not reviewed effectively, major design flaws may not be noticed, hence cause more problems during implementation.

Causes For Defect – 3) Coding Errors

- **Lack of Coding Standards**: Inconsistent coding leads to different types of errors within the code and also make maintenance process of the code difficult.
- The problem of the absence of standard checklists is that the developer can write code that is difficult to read, debug, and extend.
- **Insufficient Code Reviews:** If a proper code review is not conducted, mistakes that could be overseeing as well as other defects are likely to be spotted.

Causes For Defect – 4) Testing Gaps

- **Inadequate Testing Coverage:** This means that in cases when there are some severe faults, which can appear specifically in extreme conditions and non-functional contracts, those will not be detected in the course of shallow testing.
- **Lack of Automated Testing:** This means that the results obtained through manual testing may have errors and at the same time might not cover all the test possible scenarios.

Causes For Defect – 5) Human Factors

- **Developer Inexperience:** Fresh developers may find it hard to develop top programs since they may not have the skills and expert knowledge in order to develop good and error free code. Inexperienced developers, who assemble applications, may not know the best practices for development, which contributes to errors being made.
- **Communication Breakdowns:** Lack of communication between the members of the team affects effective project implementation since there may be confusion or misinterpretation of instructions between members.
- **Programming Errors:** Mistakes made by developers during coding, such as incorrect logic, syntax errors, or overlooked conditions.
- **Testing Errors:** Testers may miss critical scenarios, write incorrect test cases, or fail to detect defects.

Severity And Priority Of Defect - Introduction

- In software testing, a defect is the most critical entity.
- The most important attributes that can be assigned to a defect are priority and severity.
- They help teams to efficiently fix defects and go through the release scheduling processes without letting any critical issues fall through the gaps.
- In software testing, severity and priority are critical aspects of defect management.
- They help determine the defect's impact on the system and the urgency with which it should be fixed.

Severity Of Defect

- Severity is defined as the extent to which a particular defect can create an impact on the software.
- Severity is a parameter to denote the implication and the impact of the defect on the functionality of the software.
- A higher effect of the bug on system functionality will lead to a higher severity level.
- A QA engineer determines the severity level of a bug.
- **Types of Severity:**
    1. Critical
    2. Major
    3. Medium
    4. Low

Types Of Severity – 1) Critical

- A defect that has completely blocked the functionality of an application where the user or the tester cannot proceed or test anything.
- If the whole application's functionality is inaccessible or down because of a defect, such a defect is categorized as a critical defect.
- If the defect was reported in a testing environment – the testing is blocked until the defect is fixed.
- If the defect was reported in production – the users are blocked from being able to use your application thoroughly.
- **For instance–** When the login screen of an application is not working and the user cannot log in, the whole application becomes inaccessible to the user.

Types Of Severity – 2) Major

- When a bug isn't affecting the whole application but still prevents significant system functionalities from working, it becomes a Major defect.
- There won't be a complete system shutdown (which would be the case for a critical defect).
- Even so, it will prevent major, sometimes even basic, application functionalities from working.
- **For instance**, in a banking application, a user cannot transfer money to any beneficiary, but that doesn't affect their ability to add new beneficiaries.

Types Of Severity – 3) Medium

- The behavior of an application is different than expected, but this does not affect functionality.
- Usually, medium severity defects have a workaround, so they may not block a functionality completely (unlike major severity defects where there is no workaround).
- Such medium defects can wait until the next release because they do not restrict the application's functionality.
- **For instance,** the download link in the Help section of an application needs to be fixed. However, the user is still able to read the document online.

Types Of Severity – 4) Low

- Defects of cosmetic nature that don’t affect the application functionality.
- But they are valid defects nonetheless.
- **For instance,** spelling mistakes on the webpage. These are valid defects, but they can wait to be fixed since they’re not affecting application functionality.

Priority Of Defect

- Priority is defined as a parameter that decides the order in which a defect should be fixed.
- Defects having a higher priority should be fixed first.
- Defects that leave the software unstable and unusable are given higher priority over the defects that cause a small functionality of the software to fail.
- It refers to how quickly the defect should be rectified.
- **Types of Priority:**
    1. High
    2. Medium
    3. Low

Types Of Priority – 1) High

- The defect needs to be resolved immediately, as it impacts business operations or deadlines.
- These defects may affect the whole application. They need resolution as soon as possible, and assigning a high priority ensures low-resolution time.
- Usually, a high severity means a high priority as well. But this is only sometimes the case.
- **Examples**:
  - Payment gateway failure on an e-commerce website.
  - System crash in a production environment.

Types Of Priority – 2) Medium

- The defects which don't affect business and customers typically get Medium priority.
- They are less urgent than high-priority defects and can be fixed when the development team has the bandwidth to take them up.
- Such bugs can be fixed either in the same release or the next.
- **Examples**:
  - Non-critical forms not submitting correctly.
  - Features used infrequently by users.

Types Of Priority – 3) Low

- The defect is irritant but a repair can be done once the more serious defects can be fixed.
- **Examples**:
  - Typos in non-critical areas.
  - Minor layout issues in rarely used screens.

Severity And Priority Combination Examples

- **1. High Severity High Priority:**
- Consider an example of a web application where the if after filling in the login details, the user cannot click the login button then this is the case of high severity and high priority.
- **High Severity:** If the login button is not clickable then the whole application is blocked and none of the functions can be accessed by the user
- **High Priority:** If the login button is not clickable this means that the application is not letting any user log in then what is the use of such an application? Such defects are high-priority defects as the users will avoid such applications and businesses will be impacted.

Severity And Priority Combination Examples

- **2. High Severity Low Priority:**
- Consider the example of the application being used on the older versions of Internet Explorer say IE8. This is a case of high severity and low priority.
- **High Severity:** The fault, in this case, is of high severity because when the application is accessed on the older version, the page will not load properly and a few fields and text will be overlapped thus whole application will be impacted.
- **Low Priority:** The defect is of low priority because very few actual users use IE8 or older versions so the fix can wait.

Severity And Priority Combination Examples

- **3) Low Severity Low Priority:**
- Consider the example of the help or FAQ section of the website where the theme or font style of a section of the page does not match with that of the rest of the page.
- **Low Severity:** The defect is of low severity as the defect is not affecting the website functionality.
- **Low Priority:** The defect is of low priority as not many users will access this particular section of the website so the fix can wait.

Severity And Priority Combination Examples

- **4. Low Severity High Priority:**
- Consider the example when there is a typo on the website. For example, the case of the school website where the ‘Admission Form’ is misspelled as ‘Admission Form’.
- **Low Severity:** The defect is of low severity because functionality-wise there is no issue.
- **High Priority:** The defect is of high priority since it is related to business and needs to be fixed as soon as possible.

Severity V/s Priority

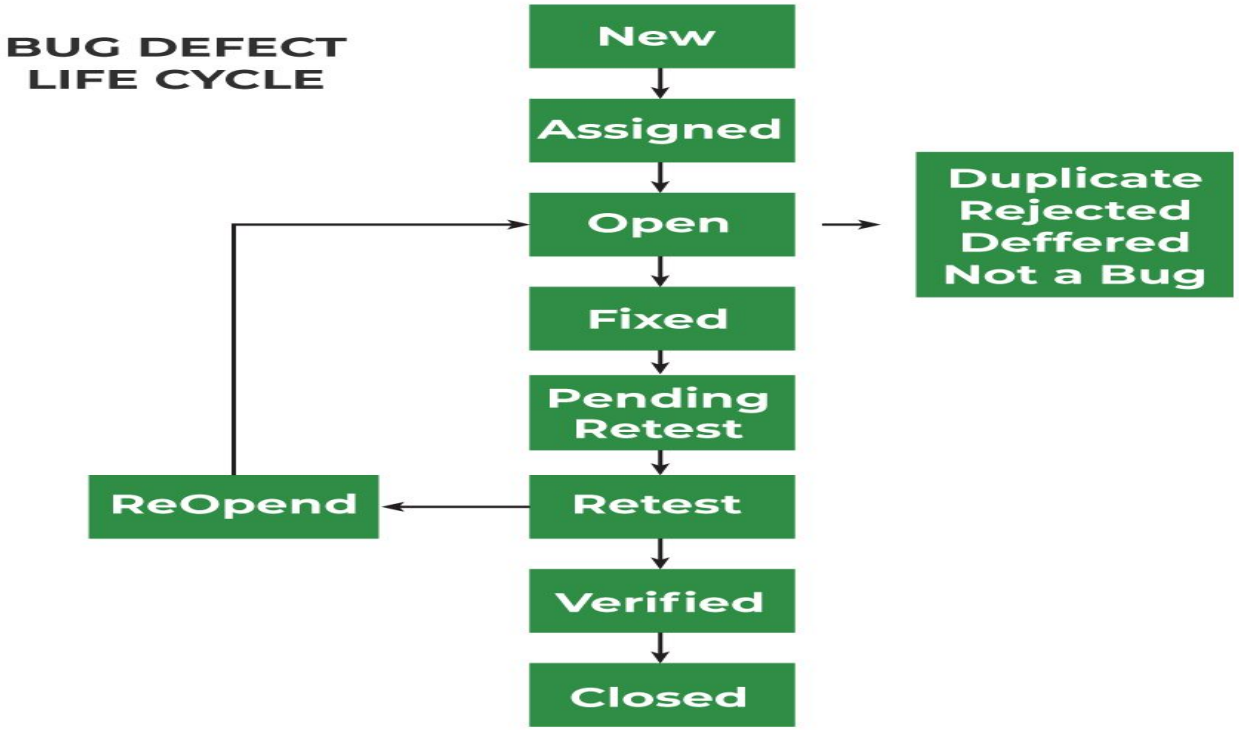
Severity	Priority
Severity is a parameter to denote the impact of a particular defect on the software.	Priority is a parameter to decide the order in which defects should be fixed.
Severity means how severe the defect is affecting the functionality.	Priority means how fast the defect has to be fixed.
Severity is related to the quality standard.	Priority is related to scheduling to resolve the problem.
Severity is divided into 4 categories: • Critical • Major • Medium • Low	Priority is divided into 3 categories: • Low • Medium • High

Severity V/s Priority

Severity	Priority
The testing engineer decides the severity level of the defect.	The product manager decides the priorities of defects.
Its value is objective.	Its value is subjective.
Its value doesn't change from time to time.	Its value changes from time to time.
It is associated with functionality or standards.	It is associated with scheduling.
It indicates the seriousness of the bug in the product functionality.	It indicates how soon the bug should be fixed.
It is driven by functionality	It is driven by business value.
It is based on the technical aspect of the product.	It is based on the customer's requirements.

Bug Life Cycle

- The Bug Life Cycle is a structured process that defines the stages a bug goes through from identification to closure in software testing.
- It ensures that bugs are tracked, fixed, and validated systematically, promoting collaboration and quality assurance.



Bug Life Cycle

- The Bug Life Cycle has following 9 phases:
- 1) New
- 2) Assigned
- 3) Open / In progress
- 4) Fixed
- 5) Pending Request
- 6) Retest
- 7) Reopen
- 8) Verified
- 9) Closed

Bug Life Cycle

- **1. New:**
- When any new bug is identified by the tester, it falls in the ‘New’ state.
- It is the first state of the Bug Life Cycle.
- The bug status is set to "New", awaiting review.
- **2. Assigned:**
- The bug is reviewed by the Test Lead, QA Manager, or Development Lead.
- If valid, it is assigned to a developer or team for investigation.
- The bug status changes to "Assigned".

Bug Life Cycle

- **3. Open:**
- In this ‘Open’ state the defect is being addressed by the developer team and the developer team works on the defect for fixing the bug.
- Based on some specific reason if the developer team feels that the defect is not appropriate then it is transferred to either the ‘Rejected’ or ‘Deferred’ state.
- **4. Fixed:**
- After necessary changes of codes or after fixing identified bug developer team marks the state as ‘Fixed’.

Bug Life Cycle

- **5. Pending Request:**
- During the fixing of the defect is completed, the developer team passes the new code to the testing team for retesting.
- And the code/application is pending for retesting on the Tester side so the status is assigned as ‘Pending Retest’.
- **6. Retest:**
- At this stage, the tester starts work of retesting the defect to check whether the defect is fixed by the developer or not, and the status is marked as ‘Retesting’.

Bug Life Cycle

- **7. Reopen:**
- After ‘Retesting’ if the tester team found that the bug continues like previously even after the developer team has fixed the bug, then the status of the bug is again changed to ‘Reopened’.
- Once again bug goes to the ‘Open’ state and goes through the life cycle again.
- This means it goes for Re-fixing by the developer team.

Bug Life Cycle

- **8. Verified:**
- The tester re-tests the bug after it got fixed by the developer team and if the tester does not find any kind of defect/bug then the bug is fixed and the status assigned is ‘Verified’.
- **9. Closed:**
- It is the final state of the Defect Cycle, after fixing the defect by the developer team when testing found that the bug has been resolved and it does not persist then they mark the defect as a ‘Closed’ state.

Bug Life Cycle – Optional Stages

- **Rejected**: If the developer or manager determines the reported issue is not valid, the bug is marked as "Rejected".
- Common reasons:
  - The issue is not reproducible.
  - It is a misunderstanding of the requirements.
- **Deferred**: If the bug is valid but deemed low priority or planned for a future release, it is marked as "Deferred".
- **Duplicate**: If the reported bug is already logged in the system, it is marked as "Duplicate".

State Of Bugs

- In software testing, bug states represent the various stages or statuses a bug goes through during its lifecycle, from identification to closure.
- Each state indicates the current position or condition of the bug in the defect management process.
- **Importance of Bug States**
- Clear Communication: Provides visibility into the progress of bug resolution.
- Efficient Tracking: Helps track and manage bugs effectively throughout their lifecycle.
- Prioritization: Ensures critical bugs are addressed first.
- Accountability: Assigns responsibility for each stage to the appropriate team members.

State Of Bugs

State	Description
New	A new bug is logged, awaiting review.
Assigned	The bug is assigned to a developer.
Open	The developer is working on fixing the bug.
Fixed	The issue is resolved, and the fix is ready for testing.
Retest	The fix is being tested by the QA team.
Verified	The bug is successfully resolved and verified.
Closed	The bug is resolved and no longer requires action.
Rejected	The bug is invalid and will not be fixed.
Deferred	The bug is valid but postponed for a future release.
Duplicate	The bug is already logged in the system.
Reopened	The bug still exists or the fix caused a new issue, requiring further action.
Not a Bug	The reported behavior is expected and not a defect.

State Of Bugs

- A bug template is a structured format used to document defects in software testing.
- It helps testers provide all the necessary information for developers to understand, reproduce, and fix the defect.
- The states of a bug remain consistent (e.g., New, Assigned, Fixed, etc.), but the template ensures proper communication of bug details.

State Of Bugs – Bug Report Template

- **Bug ID:** A unique identifier for the bug (e.g., BUG-123).
- **Bug Name:** A short headline describing the defect. It should be specific and accurate.
- **Summary:** A brief description of the bug (e.g., "Login button unresponsive").
- **Description:** A detailed explanation of the bug, including the issue , the module where it was detected and its impact.
- **Steps to Reproduce:** Step-by-step actions to reproduce the bug.
- Example: Navigate to the login page.
- Enter valid credentials.
- Click the login button.

State Of Bugs – Bug Report Template

- **Actual Result:** The behavior observed (e.g., "Login button does not respond").
- **Expected Result:** The intended behavior (e.g., "User should be redirected to the dashboard").
- **Environment Details:** Browser/OS: Chrome v100, Windows 10.
- Environment: Staging/Production.
- App version: 1.0.5.
- **Severity**: This describes the impact of the defect on the application under test.
- **Priority**: This is related to how urgent it is to fix the defect. Priority can be High/ Medium/ Low based on the impact urgency at which the defect should be fixed.

State Of Bugs – Bug Report Template

- **Attachments**: Screenshots, videos, or logs that support the report.
- **Reported By**: Name/ ID of the tester who reported the bug.
- **Date Reported:** Date when the bug was reported.
- **Status**: The current state of the bug (New, Assigned, Fixed, etc.).
- **Assigned To**: The developer or team responsible for fixing the bug.
- **Fixed By:** Name/ ID of the developer who fixed the defect.
- **Data Closed:** Date when the defect is closed.

Bug Report Template Example

Field	Details
Bug ID	BUG-101
Summary	Login button is unresponsive on the login page.
Description	The login button on the login page does not respond when clicked, preventing users from accessing their accounts.
Steps to Reproduce	1. Navigate to the login page. 2. Enter valid username and password. 3. Click the login button.
Actual Result	The login button is unresponsive and no action is triggered.
Expected Result	Clicking the login button should redirect the user to the dashboard.
Environment	Browser: Chrome v109 OS: Windows 10 Environment: Staging App Version: 2.0.1
Severity	Major
Priority	High
Attachments	Screenshot: `login_error.png`. Video: `login_bug_demo.mp4`.
Reported By	John Doe
Date Reported	2025-01-28
Status	New
Assigned To	Dev Team

Bug Report Template Example

Defect ID	D_AppXYZ_Module1_001
Defect Title	Manager Dashboard is not allowing adding newly reports types-history, work completion trends
Defect Description/Steps to reproduce	1. Login in as a manager (include manager credentials only if not confidential) 2. Click on the dashboard 3. Click on + icon to add a new report 4. Select History/Work completion report trends report type 5. Click "Add to dashboard button"
Expected Result	The new report should add and the manager should be able to view it on the dashboard
Actual Result	An error with the message "No report exists with that name" error message shows
Evidence/attachment	< Add image or a word doc with multiple images showing sequence of steps or a quick/short video >
Severity	High
Priority	Medium
Module affected	Dashboards and Manager reporting
Environment	FireFox version 32.4
Reported by	Tester XYZ
Reported on	03-02-2023
Status	New
Assigned To	Developer ABC